CS102 - Week 2 Notes Outline and Study Guide

Table of Contents

What is inside

- Week 2 overview and master framework: LIFO, stacks, push, pop, undo behavior, FIFO, queues, enqueue, dequeue, deque, queue challenges, and choosing between stacks and queues.
- Lesson 1 - What Is LIFO?
- Lesson 2 - Introduce the Stack
- Lesson 3 - Stack Operations: Push and Pop
- Lesson 4 - Building a Stack Using Lists
- Lesson 5 - Applying Stacks to Undo Behavior
- Lesson 6 - Challenge: Stacks
- Lesson 7 - What Is FIFO?
- Lesson 8 - Introduce the Queue
- Lesson 9 - Queue Operations: Enqueue and Dequeue
- Lesson 10 - Implementing Queues with deque
- Lesson 11 - Challenge: Queues
- Lesson 12 - Integrating Stacks and Queues
- Review logic, glossary, common mistakes, debugging tips, and syntax-heavy cheat sheet

Week 2 at a Glance

Week 2 builds directly from Week 1's data-structure foundation. In Week 1, the focus was how information is organized and accessed. In Week 2, the focus becomes the rule that controls which item leaves first. That rule changes the behavior of the whole structure.

The two major patterns are stacks and queues. A stack follows LIFO - Last In, First Out. A queue follows FIFO - First In, First Out. These sound like small vocabulary words, but they describe two very different ways software manages waiting items, histories, actions, and tasks.

Stacks are newest-first structures. They match plate piles, folded towels, browser back buttons, undo histories, and any situation where the most recent action should be handled first. Queues are oldest-first structures. They match checkout lines, ticket systems, print jobs, task schedulers, and any situation where fairness or arrival order matters.

Memory hook: stack = newest item leaves first. queue = oldest waiting item leaves first. If the latest action should be undone, think stack. If the earliest waiting item should be handled next, think queue.

Made with Goodnotes

Lesson 1 - What Is LIFO?

Big Idea

LIFO means Last In, First Out. The last item added to a collection is the first item removed. This lesson uses physical examples, like plates, books, apps, and folded towels, to make the rule feel obvious before turning it into code.

Full Teaching Explanation

The core idea of LIFO is order of removal. Imagine stacking clean plates in a cabinet. Every new plate goes on top. Later, when you need a plate, your hand naturally reaches for the top plate first. That top plate is the most recently added plate, so it is also the first one removed. That is Last In, First Out.

This matters because a computer science stack works the same way. The structure does not let you casually pull from the bottom or middle without disturbing the whole stack. The accessible item is the newest item at the top. LIFO is less about the object itself and more about the rule the collection follows.

A line of people is the opposite. The first person who got in line is usually the first person served. That is not LIFO because the oldest item leaves first, not the newest. Week 2 uses this contrast repeatedly: stacks remove from the newest end; queues remove from the oldest end.

The goal of this lesson is to build the mental picture first. Before you think about Python lists, append(), or pop(), you should be able to look at a situation and ask: does the newest item leave first? If yes, it probably behaves like LIFO.

Examples and Mental Models

Situation	Is it LIFO?	Why
Stack of plates	Yes	Newest plate is on top and leaves first.
Pile of books	Yes	Top book was placed last and is read first.
Folded towels	Yes	Clean towels are added on top; the top towel is taken first.
People waiting in line	No	First person in line is served first, so this is FIFO.
Closing recent apps	Yes	Most recent app or action is usually handled first.

Out Loud Programmer Thinking

When I see a collection, I ask which end matters. If the newest item is the next one removed, I am thinking in LIFO terms. If the oldest waiting item is next, I am thinking in FIFO terms.

A useful sentence is: "The last thing I put on is the first thing I take off." That sentence explains plate stacks, towel stacks, browser back buttons, and undo buttons.

Terms, Definitions, and Why They Are Called That

- LIFO: Last In, First Out. The newest item leaves first.
- Last In: the item most recently added to the structure.
- First Out: the item removed before the others.
- Top: the accessible end of a stack. New items are added at the top and removed from the top.
- Stack behavior: newest-first behavior. The structure behaves like a physical stack of objects.

Common Mistakes and Helpful Tips

- Mistake: thinking LIFO means the most important item leaves first. LIFO is about order added, not importance.
- Mistake: calling a waiting line LIFO. A normal line is FIFO because the first person in line is served first.
- Mistake: focusing on the example instead of the rule. Plates, books, towels, and apps are only examples. The rule is newest item out first.
- Helpful tip: draw arrows. Add items upward, then remove from the top. The picture usually makes the answer obvious.

Made with Goodnotes

Lesson 1 - What Is LIFO? (continued)

Notes

Made with Goodnotes

Lesson 2 - Introduce the Stack

Big Idea

A stack is a collection that follows LIFO behavior. Items are arranged so the last item added becomes the next item available at the top. A stack is not just any list of items; it is a collection with a specific access rule.

Full Teaching Explanation

A stack is the formal data-structure version of the plate pile. The top is the active end. When you add something, it goes on top. When you remove something, it comes from the top. The bottom and middle items are still in the stack, but they are not the next available item.

This is why the word stack is so visual. It describes items arranged on top of one another. In a real stack of dishes, the top item blocks access to the items below it. In a software stack, the newest item is treated as the accessible item. You can think of the stack as having a single doorway: items enter and leave through the same doorway at the top.

Stacks become useful when the program needs to remember a history of actions in reverse order. The newest action is usually the one you undo first. The most recent page might be the one a browser goes back from first. The most recent function call is the one that finishes first. All of those examples match the stack rule.

The important part is not just the definition. The important part is recognizing when a problem naturally wants stack behavior. If the task says the most recent thing should be handled first, that is a stack clue.

Stack Anatomy

Part	Meaning	Example
Top	Only directly accessible end.	Top pancake or top plate.
Bottom	Oldest item in the stack.	First plate placed down.
Push/Add	Put a new item at the top.	Add a clean towel to the stack.
Pop/Remove	Take the top item away.	Remove the top plate first.

Code Example - Thinking of a Stack as Data

Python lists can simulate stack behavior because they can add to the end and remove from the end. The end of the list acts like the top of the stack.

```
stack = []
stack.append("plate A")
stack.append("plate B")
stack.append("plate C")

print(stack)

top_plate = stack.pop()
print(top_plate)
print(stack)
```

Line-by-line explanation:

- stack = [] creates an empty list that will act like a stack.
- append() adds each new plate to the end of the list, which we treat as the top.
- pop() removes and returns the most recently added item.

Real English translation: "Build a stack of plates, then take the top plate off."

Made with Goodnotes

Lesson 2 - Introduce the Stack (continued)

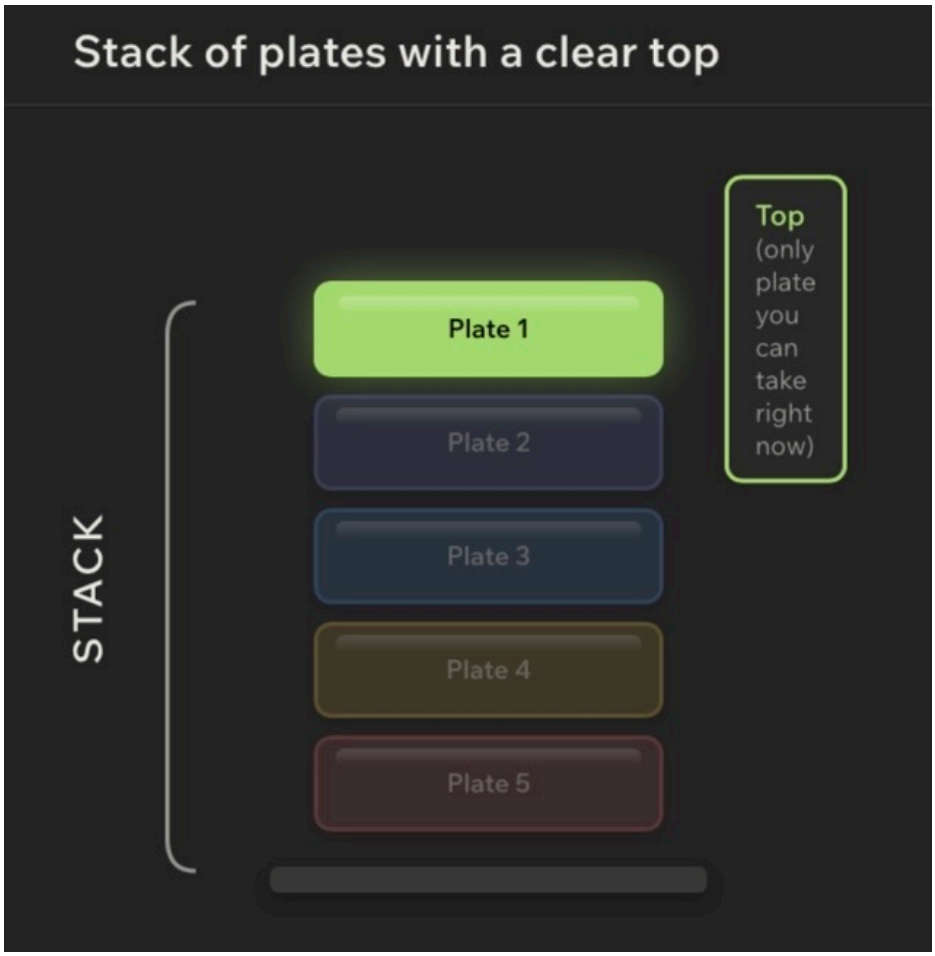
Terms, Definitions, and Why They Are Called That

- Stack: a collection where the last item added is the next item removed. It is called a stack because it behaves like objects stacked on top of one another.
- Top: the newest and next available position.
- Push: the operation that adds an item to the top. In Python list practice, append() plays this role.
- Pop: the operation that removes and returns the top item.
- LIFO: the ordering rule that makes a stack a stack.

Common Mistakes and Helpful Tips

- Mistake: thinking a stack is just any collection. A stack is defined by its removal rule.
- Mistake: trying to remove from the bottom and still call it stack behavior.
- Mistake: forgetting that the top changes every time you push or pop.
- Helpful tip: before writing code, label which side is the top. In Python list examples, we usually treat the right side/end as the top.

Notes



Made with Goodnotes

Lesson 3 - Stack Operations: Push and Pop

Big Idea

Stacks become useful when you can change them. The two core stack operations are push and pop. Push adds an item to the top. Pop removes and returns the current top item. Together, they turn LIFO from a definition into actual behavior.

Full Teaching Explanation

This lesson connects the stack concept to Python operations. In many languages, adding to a stack is called push. Removing from a stack is called pop. Python lists do not use a method named push, but list.append() behaves like push when the end of the list is treated as the stack top.

The most important detail is that pop() does two things at once. It removes the top item, and it returns that removed item. Beginners often remember the removing part but forget the returning part. That returned value matters because the program often needs to know what was just removed.

For example, if the stack contains ["A", "B", "C"], the top is "C" because it was added last. Calling pop() returns "C" and changes the stack to ["A", "B"]. If you call pop() again, it returns "B" and leaves ["A"]. This is the stack shrinking in exact reverse order.

When tracing stack code, do not only stare at the final print statement. Write the stack after each append and each pop. Stack questions are usually state questions: what does the stack contain right now?

Code Example 1 - Push Three Items, Pop One

This is the basic stack pattern used throughout the week.

```
stack = []

stack.append("A")
stack.append("B")
stack.append("C")

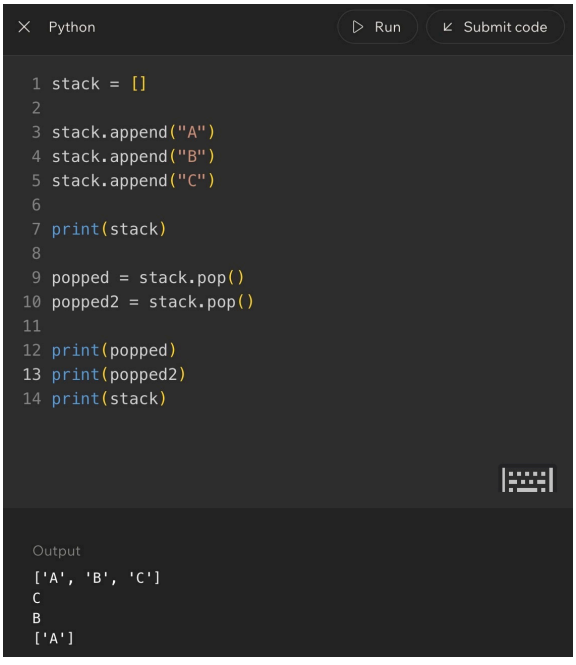
print(stack)

popped = stack.pop()
print(popped)
print(stack)
```

Line-by-line explanation:

- append("A"), append("B"), and append("C") push values onto the stack.
- After the pushes, the stack is ["A", "B", "C"]. The top is "C".
- popped = stack.pop() removes "C" and stores it in popped.
- print(popped) displays the removed item.
- print(stack) displays what is left: ["A", "B"].

Real English translation: "Add A, B, and C. Take off the top item. The top item is C."



Made with Goodnotes

Lesson 3 - Stack Operations: Push and Pop (continued)

Code Example 2 - Pop Twice

After one pop, the item underneath becomes the new top.

```
stack = []
stack.append("A")
stack.append("B")
stack.append("C")

popped = stack.pop()
popped2 = stack.pop()

print(popped)
print(popped2)
print(stack)
```

Output logic:

- The first pop returns "C".
- The second pop returns "B" because after C leaves, B becomes the top.
- The stack ends as ["A"].

Out loud programmer thinking: "The top keeps changing after each pop."

Stack State Trace

Operation	Returned Value	Stack After
Start	-	[]
append("A")	-	["A"]
append("B")	-	["A", "B"]
append("C")	-	["A", "B", "C"]
pop()	"C"	["A", "B"]
pop()	"B"	["A"]

Terms, Common Mistakes, and Helpful Tips

- Push: add an item to the top of the stack. In Python list examples, append() does this.
- Pop: remove and return the item at the top. In Python lists, pop() does this from the end by default.
- Returned value: the value pop() gives back after removing it.
- Mistake: thinking pop() only deletes. It also returns the removed item.
- Mistake: losing track of the new top after a pop.
- Tip: write the stack after every operation when solving quiz questions.

Notes

Made with Goodnotes

Lesson 4 - Building a Stack Using Lists

Big Idea

Python lists can be used to build stack behavior when you consistently use append() to push items and pop() to remove the most recent item. The list is the storage tool; the stack is the behavior pattern you enforce.

Full Teaching Explanation

This lesson turns the stack idea into a repeatable coding pattern. Start with an empty list. Use append() to add items. Use pop() to undo or remove the newest item. After every operation, the list represents the current stack state.

The key is consistency. A list can do many things besides stack behavior. It can insert in the middle, remove by value, sort, slice, and more. But when you are using it as a stack, you intentionally restrict yourself to stack-style operations. You push to the end and pop from the end.

This is an important programming habit: sometimes the data structure is about how you choose to use the tool. Python does not stop you from misusing the list, so you have to preserve the stack rule yourself.

Another important idea is empty-stack safety. If you call pop() on an empty list, Python raises an error because there is no top item to remove. Before popping, you should check whether the stack has anything in it. That guard turns fragile code into safer code.

Code Example 1 - Build and Shrink a Stack

This example shows the stack growing with append() and shrinking with pop().

```
stack = []

stack.append("box1")
print(stack)

stack.append("box2")
print(stack)

stack.append("box3")
print(stack)

top_box = stack.pop()
print(top_box)
print(stack)
```

Real English translation:

- Start with no boxes.
- Add box1, then box2, then box3.
- The top box is box3 because it was added last.
- pop() removes box3 and returns it into top_box.
- The remaining stack is ["box1", "box2"].

Code Example 2 - Checking Before Popping

Before popping, check whether the stack contains at least one item.

```
stack = []

if len(stack) > 0:
    item = stack.pop()
    print(item)
else:
    print("empty")
```

Line-by-line explanation:

- len(stack) > 0 checks whether there is an item to remove.
- If the stack has something, pop() is safe.
- If the stack is empty, the else branch runs instead.

Real English translation: "Only remove from the stack if there is actually something there."

Made with Goodnotes

Lesson 4 - Building a Stack Using Lists (continued)

Trace Table - Mini Challenge

Operation	What It Does	Why It Helps
append("item1")	Adds item1 to the top.	Shows stack growth.
append("item2")	Adds item2 above item1.	Newest item becomes top.
pop()	Removes item2.	Newest item leaves first.
len(stack) > 0	Checks for items.	Prevents empty pop errors.

Terms, Common Mistakes, and Helpful Tips

- Empty stack: a stack with no items left to remove.
- Guard condition: a condition that protects code from unsafe behavior.
- State: the current contents of the stack at a specific moment.
- Mistake: calling pop() repeatedly without checking whether anything is left.
- Mistake: assuming the original stack stays the same after pop(). pop() changes the stack.
- Tip: after each stack operation, write the new stack state in brackets.

Notes

Code Example:

Python ▷ Run ↙ Submit code

```
1  stack = []
2
3  stack.append("box1")
4  print(stack)
5
6  stack.append("box2")
7  print(stack)
8
9  stack.append("box3")
10 print(stack)
11
12 top_box = stack.pop()
13
14 print(top_box)
15 print(stack)
```

Output

```
['box1']
['box1', 'box2']
['box1', 'box2', 'box3']
box3
['box1', 'box2']
```

Made with Goodnotes

Code Example:

Python Run Submit code

```python
stack = []

stack.append("item1")
stack.append("item2")
stack.append("item3")


stack.pop()
print(stack)

stack.pop()
print(stack)

if len(stack) > 0:
    stack.pop()
    print(stack)

if len(stack) == 0:
    print("Done, stack is empty")
```



Output

```
['item1', 'item2']
['item1']
[]
Done, stack is empty
```

Made with Goodnotes

Lesson 5 - Applying Stacks to Undo Behavior

Big Idea

Undo behavior is one of the clearest real-world uses of a stack. Each action is pushed onto a history stack. When the user presses undo, the most recent action is popped first.

Full Teaching Explanation

Imagine drawing three shapes in order: a red line, a blue circle, and a green square. If you press undo once, the app removes the green square first because that was the most recent action. If you press undo again, it removes the blue circle. The actions are undone in reverse order.

That reverse-order behavior is exactly LIFO. The first action may still matter, but it is buried under newer actions. The newest action is the one sitting on top of the stack. Undo systems use this idea because users usually expect the last thing they did to be the first thing removed.

In code, you can model an undo history with a list. Each user action is appended to the stack. When undo happens, pop() removes the latest action and returns it so the program knows what to reverse. The remaining list represents the actions that still exist.

The deeper lesson is that stacks are not abstract trivia. They model real software memory. Whenever a program needs to remember a sequence and walk backward through it, a stack is often the right mental model.

Code Example 1 - Undo Type Actions

The list stores action history. Each append() records a new action. Each pop() undoes the latest action.

```
actions = []

actions.append("open file")
actions.append("type hello")
actions.append("type world")
actions.append("press enter")
actions.append("type !")

last_action = actions.pop()
print(last_action)
print(actions)
```

Line-by-line explanation:

- actions = [] creates the undo history.
- Each append() records what the user just did.
- The final action, "type !", is the top of the stack.
- actions.pop() removes and returns that latest action.

Real English translation: "Remember everything the user did, then undo the newest action first."

Undo State Evolution

Action	Stack State	Meaning
Open file	["open file"]	One action recorded.
Type hello	["open file", "type hello"]	New action added on top.
Type world	[..., "type world"]	History keeps growing.
Undo once	Removes "type world"	Most recent action is reversed.
Undo again	Removes "type hello"	Next most recent action is reversed.

Made with Goodnotes

Lesson 5 - Applying Stacks to Undo Behavior (continued)

Code Example 2 - Save the Undo Action

Usually, the program needs to know which action was undone.

```
actions = ["draw line", "draw circle", "draw square"]

undo_1 = actions.pop()
undo_2 = actions.pop()

print(undo_1)
print(undo_2)
print(actions)
```

Output logic:

- undo_1 is "draw square" because it was added last.
- undo_2 is "draw circle" because after square is removed, circle becomes the top.
- The remaining stack is ["draw line"].

This is the same LIFO pattern from plates, books, and towels, but now it powers an actual software feature.

Real-World Meaning

Stacks appear anywhere software needs to move backward through recent history. Undo buttons, browser back buttons, editor action histories, and some navigation flows all depend on the idea that the latest action is the first one reversed.

This is why the stack rule matters. It does not just describe storage. It describes what users expect. If undo removed the oldest action first, the feature would feel broken even if the code technically ran.

Terms, Common Mistakes, and Helpful Tips

- Undo stack: a stack that stores actions so the newest action can be reversed first.
- Action history: the ordered list of actions a user performed.
- Current state: what remains after some actions are undone.
- Mistake: undoing the first action instead of the latest action.
- Mistake: popping from an empty undo stack.
- Tip: when you see "undo," think "store actions as they happen, then pop the newest one."

Key Takeaways

- LIFO means the last item added is the first item removed.
- A stack is a data structure that enforces LIFO behavior.
- push adds to the top; pop removes and returns the top.
- Python lists can model stacks with append() and pop().
- Undo behavior is a stack because the most recent action is undone first.
- Always trace stack state after each operation, especially after pop().

Notes

Made with Goodnotes

Code Example:

Python Run Submit code

```python
actions = []

actions.append("open file")
actions.append("type 'hello'")
actions.append("type ' world'")
actions.append("press enter")
actions.append("type '!'")

actions.pop()
actions.pop()

print(actions)
```

Output

```
['open file', "type 'hello'", "type ' world'"]
```

Made with Goodnotes

Lesson 6 - Challenge: Stacks

Big Idea

This lesson is where the stack idea stops being just a definition and becomes a tracing skill. You practice reading a sequence of push and pop operations, predicting which value is removed, and describing what remains in the stack afterward.

The important habit is not guessing. The important habit is tracking the stack state after every line. In stack problems, order matters because the last item pushed is the first item available to pop.

Full Teaching Explanation

A stack is easiest to understand when you treat it like a physical pile. Every new item goes onto the top. When something is removed, it comes off the top. That means the newest item is the most accessible item. The bottom items are still there, but you cannot reach them until the items above them are removed first.

In Python, a list can simulate a stack. The right side of the list acts like the top of the stack. When you use append(), Python adds the new item to the end of the list. When you use pop(), Python removes and returns the last item from the list. That gives you LIFO behavior: Last In, First Out.

This lesson uses challenge rounds because stacks are less about memorizing words and more about being able to trace motion. Each append changes the stack by adding a new top. Each pop changes the stack by removing the current top. If the popped value is stored in a variable, that variable remembers the removed value while the stack itself has already changed.

Stack State Mental Model

Question	What to Ask Yourself
What is on top?	Look at the most recently appended item, usually the rightmost item in a Python list.
What does pop() remove?	It removes and returns the current top item.
What remains?	Everything underneath the removed item stays in the stack.
Did a variable store the pop?	The variable stores the removed value; the stack no longer contains it.

Made with Goodnotes

Lesson 6 - Challenge: Stacks (Code Walkthrough)

Code Example 1 - One Pop Removes the Top

```
stack = []
stack.append("plate A")
stack.append("plate B")
stack.append("plate C")

removed = stack.pop()

print(removed)
print(stack)
```

Expected output:

```
plate C
['plate A', 'plate B']
```

Line-by-line explanation:

- stack = [] starts with an empty stack.
- append('plate A'), append('plate B'), and append('plate C') push three plates onto the stack in that order.
- Because plate C was added last, it is on top.
- removed = stack.pop() removes plate C and stores that removed value in removed.
- After the pop, the stack still contains plate A and plate B, with plate B now on top.

Real English translation: Build a stack of plates, take the top plate off, remember which plate you took, and then look at what is left.

Code Example 2 - Multiple Pops

```
stack = []
stack.append(10)
stack.append(20)
stack.append(30)

first = stack.pop()
second = stack.pop()

print(first)
print(second)
print(stack)
```

Expected output:

```
30
20
[10]
```

The first pop removes 30 because 30 was added last. After 30 is gone, 20 becomes the new top. The second pop removes 20. The only value left at the bottom is 10.

Out loud programmer thinking: 'I do not just ask what values were added. I ask what the stack looks like after each operation.'

Made with Goodnotes

Lesson 6 - Challenge: Stacks (Debug Thinking)

Terms, Definitions, and Why They Matter

- Push: adding an item to the top of a stack. In Python list stack examples, append() performs the push.
- Pop: removing and returning the top item. pop() changes the list and gives back the removed value.
- Top: the next item that will be removed. In these Python list examples, the top is the rightmost item.
- Bottom: the oldest item in the stack. It cannot be removed until everything above it is gone.
- Stack state: the current contents of the stack after some operations have already happened.
- Mutation: changing the collection itself. pop() mutates the stack because the removed item is no longer inside it.

Common Mistakes and Helpful Tips

- Mistake: thinking pop() only shows the top item. It does not just peek. It removes the item.
- Mistake: forgetting that a variable can store the popped value. If first = stack.pop(), first gets the removed item.
- Mistake: reading the list left-to-right as if the left side is always the top. In the course examples, the right side is the top.
- Mistake: trying to pop from an empty stack. If there are no items, pop() has nothing to remove and Python can raise an error.
- Helpful tip: after every append or pop, rewrite the stack state. It feels slow, but it prevents almost every stack-tracing mistake.
- Memory hook: append grows the top, pop removes the top, and the top is always the newest item.

Notes

Made with Goodnotes

Lesson 7 - What Is FIFO?

Big Idea

FIFO means First In, First Out. The first item that enters a collection is the first item removed. If LIFO feels like a stack of plates, FIFO feels like a line of people waiting their turn.

This lesson matters because it introduces the second major access pattern of the week. Stacks are useful when the newest item should be handled first. Queues are useful when the oldest waiting item should be handled first.

Full Teaching Explanation

A FIFO structure preserves arrival order. Imagine a coffee shop line. The person who arrived first stands at the front, waits the longest, and gets served first. New people join at the back. Nobody should cut ahead of the people already waiting. That is the entire idea behind FIFO.

This is different from LIFO. In LIFO, the most recently added item is the next one removed. In FIFO, the oldest waiting item is the next one removed. Both patterns are about order of removal, but they answer opposite questions. LIFO asks, 'What was added most recently?' FIFO asks, 'Who has been waiting the longest?'

The lesson uses lines, restaurant orders, general admission events, shopping app orders, and service systems because those examples all rely on fairness or chronological order. The first request, order, person, or task should usually be handled before newer ones.

Examples and Mental Models

Situation	FIFO?	Why
Coffee shop line	Yes	The first person in line gets served first.
Restaurant order queue	Yes	Orders are prepared in the order they arrive.
General admission event	Usually yes	People line up and enter in arrival order unless priority rules exist.
Shopping app orders	Often yes	Earlier orders are usually handled before later orders.
Stack of books	No	That is usually LIFO because the top/newest item is removed first.

Made with Goodnotes

Lesson 7 - FIFO vs. LIFO

Direct Comparison

Pattern	Rule	Mental Model	Best When...
LIFO	Last In, First Out	Stack of plates	The newest item should be handled first.
FIFO	First In, First Out	Line at a coffee shop	The oldest waiting item should be handled first.

Real English translation: A stack says, 'take the newest thing.' A queue says, 'serve the person or item that has been waiting the longest.'

Out loud programmer thinking: 'Before I choose a structure, I need to know which end matters. Am I handling the newest item first or the oldest waiting item first?'

Terms, Mistakes, and Helpful Tips

- FIFO: First In, First Out. The first item added is the first item removed.
- Queue: a collection that follows FIFO behavior. New items join at the back, and items leave from the front.
- Front: the end where items leave. In a line, this is where service happens.
- Rear/back: the end where new items enter.
- Mistake: calling any ordered collection FIFO. FIFO is specifically about removal order, not just having an order.
- Mistake: confusing fairness with stack behavior. A fair line is FIFO, not LIFO.
- Memory hook: FIFO is the polite line. First person waiting gets helped first.

Notes

Made with Goodnotes

Lesson 8 - Introduce the Queue

Big Idea

A queue is the data structure version of a line. New items are added at the rear, and items are removed from the front. That makes a queue the natural structure for FIFO behavior.

This lesson takes the idea from Lesson 7 and gives it a computer science name. When the first item in should be the first item out, you are thinking in queue terms.

Full Teaching Explanation

A queue is an ordered collection with two important ends: the front and the rear. The front is where items leave. The rear is where items enter. In a coffee shop, the customer at the front is served next, and new customers join at the back.

The course examples use mobile coffee orders to make this concrete. New drinks get added at the back of the line, customers take their drinks from the front, and the drink that has been waiting the longest leaves first. The order is not random, and it is not newest-first. It is FIFO: First In, First Out.

In Python, an introductory queue can be modeled with a list. The end of the list can act as the rear where append() adds new items. The beginning of the list, index 0, can act as the front where pop(0) removes the oldest item. This is simple and clear for learning the concept. Later, Python's deque gives a more efficient queue implementation.

Queue Direction Model

Queue Part	Meaning	Python List Learning Model
Rear/back	Where new items join	queue.append(item)
Front	Where the next item leaves	queue.pop(0)
Oldest item	The item waiting the longest	The value at index 0
Newest item	The item most recently added	The rightmost value

Made with Goodnotes

Lesson 8 - Introduce the Queue (Code Walkthrough)

Code Example - Enqueue and Dequeue with a List

```
queue = []

queue.append("order A")
queue.append("order B")
queue.append("order C")

served = queue.pop(0)

print(served)
print(queue)
```

Expected output:

```
order A
['order B', 'order C']
```

Line-by-line explanation:

- queue = [] starts with an empty line.
- append('order A'), append('order B'), and append('order C') add new orders to the rear of the queue.
- Because order A was added first, it is at the front.
- served = queue.pop(0) removes and returns the item at index 0, which is the front of the queue.
- After order A leaves, order B becomes the new front.

Real English translation: Build a line of orders, serve the oldest order first, and leave the remaining orders waiting in the same order.

Common Mistakes and Helpful Tips

- Mistake: using pop() instead of pop(0). pop() removes from the end, which creates stack-like LIFO behavior instead of queue-like FIFO behavior.
- Mistake: thinking the rear and front are the same end. Queues use two ends: add at the rear, remove from the front.
- Mistake: assuming the middle item leaves next. In a standard queue, the front item leaves next.
- Helpful tip: draw queues horizontally. Put FRONT on the left and REAR on the right so pop(0) and append() make visual sense.
- Memory hook: queue = line. New people join the back. The front person leaves first.

Notes

Made with Goodnotes

Lesson 9 - Queue Operations: Enqueue and Dequeue

Big Idea

Queues follow FIFO: First In, First Out. This lesson turns that idea into actual Python behavior by showing how to add items to the rear of a queue and remove items from the front.

The main operations are enqueue and dequeue. Enqueue means adding a new item to the back of the line. Dequeue means removing and returning the item at the front of the line.

Full Teaching Explanation

A queue is a data structure for waiting turns. The first item added is the first item removed. If a stack feels like a pile of plates, a queue feels like a food-truck line, checkout line, ticket window, or customer-service queue.

In the early examples, the lesson models a queue with a regular Python list. The rear of the queue is the end of the list, so append() works for adding new people to the back. The front of the queue is index 0, so pop(0) removes the oldest item. That matches FIFO because the item that has been waiting the longest leaves first.

This lesson also introduces an important safety habit: do not dequeue from an empty queue. Before removing from the front, check whether the queue still has items. In plain English, if there is no one in line, you should not try to serve the next person.

Queue Vocabulary

Term	Meaning	Python List Version
Front	The next item to be removed	Index 0
Rear	Where new items are added	End of the list
Enqueue	Add an item to the rear	queue.append(item)
Dequeue	Remove and return the front item	queue.pop(0)
Empty queue	A queue with no items waiting	len(queue) == 0

Code Example:

Python — Run — Submit code

```
queue = []

queue.append("item1")
queue.append("item2")
queue.append("item3")

if len(queue) > 0:
    dequeue = queue.pop(0)

print(dequeue)
print(queue)
```

Output

```
item1
['item2', 'item3']
```

Made with Goodnotes

Lesson 9 - Queue Operations: Enqueue and Dequeue (Code)

Code Example 1 - Basic Enqueue and Dequeue

```
queue = ["Alice", "Bob", "Charlie"]

queue.append("Daisy")      # enqueue at the rear
served = queue.pop(0)      # dequeue from the front

print(served)
print(queue)
```

Expected output:

```
Alice
['Bob', 'Charlie', 'Daisy']
```

Line-by-line explanation:

- queue starts with Alice at the front, then Bob, then Charlie.
- queue.append('Daisy') adds Daisy to the rear. She does not cut in front of anyone.
- served = queue.pop(0) removes Alice because Alice is at index 0, the front of the line.
- The remaining queue starts with Bob, then Charlie, then Daisy.

Real English translation: Add Daisy to the back of the line, serve the person at the front, and then show who is still waiting.

Code Example 2 - Functions for Queue Behavior

```
def enqueue(queue, item):
    queue.append(item)


def dequeue(queue):
    if len(queue) > 0:
        return queue.pop(0)
    else:
        return None

queue = []
enqueue(queue, "Alice")
enqueue(queue, "Bob")
enqueue(queue, "Charlie")

served = dequeue(queue)
print(served)
print(queue)
```

Expected output:

```
Alice
['Bob', 'Charlie']
```

Why this version is cleaner: the behavior now has names. enqueue() means add to the rear. dequeue() means remove from the front if there is anyone in line. The if len(queue) > 0 check prevents the function from trying to remove from an empty queue.

Made with Goodnotes

Lesson 9 - Queue Operations: Debug Thinking

Line-by-Line Programmer Thinking

- Before append: ask who is already waiting.
- After append: place the new item at the rear, not the front.
- Before pop(0): check whether the queue has at least one item.
- After pop(0): the old front is gone, and everyone behind that item shifts forward.
- If the queue is empty: return None or handle the situation safely instead of crashing.

Common Mistakes and Helpful Tips

- Mistake: using pop() instead of pop(0). pop() removes from the end, which behaves like a stack, not a queue.
- Mistake: adding new items to the front. In a normal queue, new arrivals go to the rear.
- Mistake: forgetting that pop(0) changes the queue. The removed item is no longer inside the list.
- Mistake: trying to dequeue from an empty queue. Always check len(queue) first when the queue might be empty.
- Helpful tip: draw the queue like a line: front on the left, rear on the right. It makes the direction of movement obvious.
- Memory hook: enqueue enters at the rear; dequeue departs from the front.

Notes

Code Example:

Python — Run — Submit code

```
def enqueue(queue, item):
    queue.append(item)

def dequeue(queue):
    if len(queue) > 0:
        return queue.pop(0)
    else:
        return None

queue = []

enqueue(queue, "Alice")
enqueue(queue, "Bob")
enqueue(queue, "Charlie")

served = dequeue(queue)

print(served)
print(queue)
```

Output

```
Alice
['Bob', 'Charlie']
```

Python — Run — Submit code

```
def enqueue(queue, item):
    queue.append(item)

def dequeue(queue):
    if len(queue) > 0:
        return queue.pop(0)
    else:
        return None

queue = []

enqueue(queue, "Mia")
enqueue(queue, "Noah")
enqueue(queue, "Liam")

served1 = dequeue(queue)
print(served1)

served2 = dequeue(queue)
print(served2)

served3 = dequeue(queue)
print(served3)

served4 = dequeue(queue)
if served4 == None:
    print("No one left in line")
```

Output

```
Mia
Noah
Liam
No one left in line
```

Made with Goodnotes

Lesson 10 - Implementing Queues with deque

Big Idea

This lesson improves the queue implementation. Instead of using a regular list with pop(0), Python provides deque, a special structure designed for fast additions and removals from both ends.

The queue rule does not change. Items still enter at the rear and leave from the front. What changes is the tool Python uses to make that behavior cleaner and more efficient.

Full Teaching Explanation

A regular Python list can model a queue, but it is not the ideal tool when you need to remove from the front repeatedly. With a list, pop(0) removes the first item, but the remaining items must shift forward. That can become inefficient as the collection grows.

The deque type comes from Python's collections module. The name stands for double-ended queue. It is built to add and remove items efficiently from either end. For FIFO queue behavior, you add to the rear with append() and remove from the front with popleft().

The key concept is that deque lets the code say what it means. append() still means add to the back. popleft() makes the front-removal behavior explicit. Instead of using pop(0), you can read popleft() and immediately understand that the left/front item is leaving.

List Queue vs. deque Queue

Operation	List Version	deque Version	Meaning
Create queue	queue = []	queue = deque()	Start empty
Add to rear	queue.append(item)	queue.append(item)	New item joins back
Remove front	queue.pop(0)	queue.popleft()	Oldest item leaves
Check empty	len(queue) > 0	len(queue) > 0	Remove only if safe

Code Example:

Python

Run Submit code

```
1 from collections import deque
2
3 queue = deque()
4
5 queue.append("Sophia")
6 queue.append("CJ")
7 queue.append("Caleb")
8
9 person_done = queue.popleft()
10
11 print(person_done)
12 print(queue)
```

Output

```
Sophia
deque(['CJ', 'Caleb'])
```

Python

Run Submit code

```
1 from collections import deque
2
3 # create an empty queue
4 queue = deque()
5
6 # add to the REAR (back)
7 queue.append("A")
8 queue.append("B")
9
10 # remove from the FRONT
11 person = queue.popleft()
12 print(person)   # "A"
13 print(queue)    # deque(['B'])
```

Output

```
A
deque(['B'])
```

Made with Goodnotes

Lesson 10 - Implementing Queues with deque (Code)

Code Example 1 - Basic deque Queue

```
from collections import deque

queue = deque()

queue.append("A")
queue.append("B")

person = queue.popleft()

print(person)
print(queue)
```

Expected output:

```
A
deque(['B'])
```

Line-by-line explanation:

- from collections import deque imports the queue-friendly structure.
- queue = deque() creates an empty deque.
- queue.append('A') and queue.append('B') add items to the rear.
- queue.popleft() removes the item from the front, so A leaves first.
- The queue still contains B afterward.

Real English translation: Build a line, let the first person leave, and show who remains in line.

Code Example 2 - Safe deque Removal

```
from collections import deque

queue = deque()
queue.append("Sophia")
queue.append("CJ")
queue.append("Caleb")

if len(queue) > 0:
    person_done = queue.popleft()
else:
    person_done = None
    print("No one left in queue")

print(person_done)
print(queue)
```

Expected output:

```
Sophia
deque(['CJ', 'Caleb'])
```

The if statement is the safety guard. It checks whether the queue has someone in it before calling popleft(). That matters because popleft() needs a front item to remove. If the queue is empty, there is no front item, so the safe branch stores None instead.

Made with Goodnotes

Lesson 10 - Implementing Queues with deque (Practice Pattern)

Code Example 3 - Removing Two People Safely

```
from collections import deque

ride_line = deque()
ride_line.append("Sophia")
ride_line.append("CJ")
ride_line.append("Caleb")

if len(ride_line) > 0:
    person1 = ride_line.popleft()
else:
    person1 = None
    print("No one left in queue")

if len(ride_line) > 0:
    person2 = ride_line.popleft()
else:
    person2 = None
    print("No one left in queue")

print("Left the line:", person1)
print("Left the line:", person2)
print("Still in line:", ride_line)
```

Expected output:

```
Left the line: Sophia
Left the line: CJ
Still in line: deque(['Caleb'])
```

This example repeats the safe check before each removal. That matters because the queue changes after every popleft(). A queue that was not empty before the first removal might become empty before the second removal.

Terms, Common Mistakes, and Helpful Tips

- deque: a double-ended queue from Python's collections module. It is designed for efficient work at both ends.
- popleft(): removes and returns the item on the left/front side of the deque.
- append(): adds an item to the right/rear side of the deque.
- Mistake: forgetting to import deque before using it.
- Mistake: using pop() when FIFO behavior requires popleft(). pop() removes from the rear.
- Tip: if you are modeling a real line, deque is usually the better tool than a plain list.
- Memory hook: append to the back, popleft from the front.

Notes

Made with Goodnotes

Code Example:

Python

Run | Submit code

```python
from collections import deque

queue = deque()

queue.append("Sophia")
queue.append("CJ")
queue.append("Caleb")

if len(queue) > 0:
    person_done = queue.popleft()
else:
    person_done = None
    print("No one left in queue")

print(person_done)
print(queue)
```

Output

```
Sophia
deque(['CJ', 'Caleb'])
```

Python

Run | Submit code

```python
from collections import deque

ride_line = deque()

ride_line.append("Sophia")
ride_line.append("CJ")
ride_line.append("Caleb")

if len(ride_line) > 0:
    person1 = ride_line.popleft()
    person2 = ride_line.popleft()
else:
    print("No one left in queue")

print("Left the line:", person1)
print("Left the line:", person2)
print("Still in line:", ride_line)
```

Output

```
Left the line: Sophia
Left the line: CJ
Still in line: deque(['Caleb'])
```

Python

Run | Submit code

```python
from collections import deque

ride_line = deque()

ride_line.append("Sophia")
ride_line.append("CJ")
ride_line.append("Caleb")

if len(ride_line) > 0:
    person1 = ride_line.popleft()
else:
    person1 = None
    print("No one left in queue")

if len(ride_line) > 0:
    person2 = ride_line.popleft()
else:
    person2 = None
    print("No one left in queue")

print("Left the line:", person1)
print("Left the line:", person2)
print("Still in line:", ride_line)
```

Output

```
Left the line: Sophia
Left the line: CJ
Still in line: deque(['Caleb'])
```

Made with Goodnotes

Lesson 11 - Challenge: Queues

Big Idea

This lesson gives you practice tracing queues the same way the stack challenge gave you practice tracing stacks. The goal is to predict what popleft() returns, what remains in the queue, and how repeated removals behave.

The big rule is simple but strict: with a queue, the first item added is the first item removed. Every challenge is really testing whether you can keep that FIFO order straight.

Full Teaching Explanation

Queue challenges are about state over time. The first line creates the queue. Each append adds a new item at the rear. Each popleft removes the current front item. After a removal, the second item becomes the new front. That means you cannot solve queue problems by only looking at the final line of code. You have to trace the queue after each operation.

The lesson also introduces loops with queues. A while loop can keep removing items until the queue is empty. This is useful in real programs because queues often represent work waiting to be handled: messages, jobs, tickets, people, downloads, orders, or tasks. The program keeps processing until there is nothing left.

The danger is trying to remove from an empty queue. A while len(queue) > 0 pattern is safe because it only runs while at least one item exists. That condition protects the popleft() call.

Queue Challenge Mental Model

Pattern	Meaning	What to Watch
first = q.popleft()	Remove the oldest item	first stores the removed front value
second = q.popleft()	Remove the new oldest item	The queue already changed after first
while q:	Repeat while items remain	The loop stops when the queue is empty
len(q) > 0	Safety check	Prevents empty-queue removal

Made with Goodnotes

Lesson 11 - Challenge: Queues (Code)

Code Example 1 - Two popleft Calls

```
from collections import deque

q = deque()
q.append(10)
q.append(20)
q.append(30)

first = q.popleft()
second = q.popleft()

print(first)
print(second)
print(q)
```

Expected output:

```
10
20
deque([30])
```

Line-by-line explanation:

- 10 enters first, so it is at the front.
- 20 enters second, behind 10.
- 30 enters third, at the rear.
- first = q.popleft() removes 10.
- second = q.popleft() removes 20, because 20 becomes the new front after 10 leaves.
- The queue still contains 30.

Real English translation: Serve the first two people in line and show who is still waiting.

Code Example 2 - Draining a Queue

```
from collections import deque

q = deque()
q.append("first")
q.append("second")
q.append("third")

while q:
    item = q.popleft()
    print(item)
```

Expected output:

```
first
second
third
```

The while q loop keeps going as long as the queue has at least one item. Each loop removes one item from the front. The output appears in FIFO order because popleft() always removes the oldest remaining item.

Made with Goodnotes

Lesson 11 - Challenge: Queues (Mastery Check)

Why the Loop Pattern Matters

A queue often represents work waiting to be processed. The loop pattern is how a program handles every item without knowing ahead of time how many items there will be. The code does not need to know whether the queue contains 3 items, 30 items, or 300 items. It can simply keep processing while the queue is not empty.

This is also why the lesson emphasizes safe queue logic. If the loop condition is correct, popleft() only runs when there is something available to remove. If the condition is missing or wrong, the code may try to remove from an empty queue and crash.

Common Mistakes and Helpful Tips

- Mistake: assuming popleft() always returns the same value. It returns the current front, and the front changes after each removal.
- Mistake: forgetting that while q becomes false when the queue is empty.
- Mistake: using append and pop together, which creates stack-like behavior instead of queue behavior.
- Mistake: tracing the items but not the order they entered.
- Helpful tip: label the front and rear before solving any queue trace.
- Memory hook: queues process the oldest waiting item first.

Notes

Made with Goodnotes

Lesson 12 - Integrating Stacks and Queues

Big Idea

This lesson combines both structures. A queue controls the order of who gets processed first, and a stack can control the order of items attached to that person or task.

The point is not that one structure is always better. The point is that different parts of the same program may need different order rules.

Full Teaching Explanation

Stacks and queues answer different order questions. A queue answers, 'Who has been waiting the longest?' A stack answers, 'What was added most recently?' In real software, both questions can matter at the same time.

The lesson uses an adventure-style example. Heroes wait in a line, so the heroes are stored in a queue. The first hero in line should go first. But each hero has a backpack of items, and a backpack behaves like a stack. The item packed last is the item that comes out first.

This is a powerful mental model because it shows that data structures can be nested inside a bigger process. The outer process might be FIFO, while an inner collection behaves like LIFO. Professional programming is often about choosing the right structure for each part of the problem.

Choosing the Right Structure

Part of the Problem	Best Structure	Why
Heroes waiting their turn	Queue	First hero in line should go first
Items inside each backpack	Stack	Last packed item comes out first
Undo history	Stack	Most recent action is undone first
Ticket processing	Queue	Oldest ticket should usually be handled first

Made with Goodnotes

Lesson 12 - Integrating Stacks and Queues (Code)

Code Example 1 - Queue of Heroes, Stack of Items

```
from collections import deque

heroes_line = deque()

heroes_line.append(("Aria", ["sword", "map"]))
heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
heroes_line.append(("Cora", ["gem", "shield", "axe"]))

print(heroes_line)

current_hero, backpack = heroes_line.popleft()
print("Current hero:", current_hero)
print("Starting backpack:", backpack)

while backpack:
    item = backpack.pop()
    print(current_hero, "uses", item)
```

Expected idea:

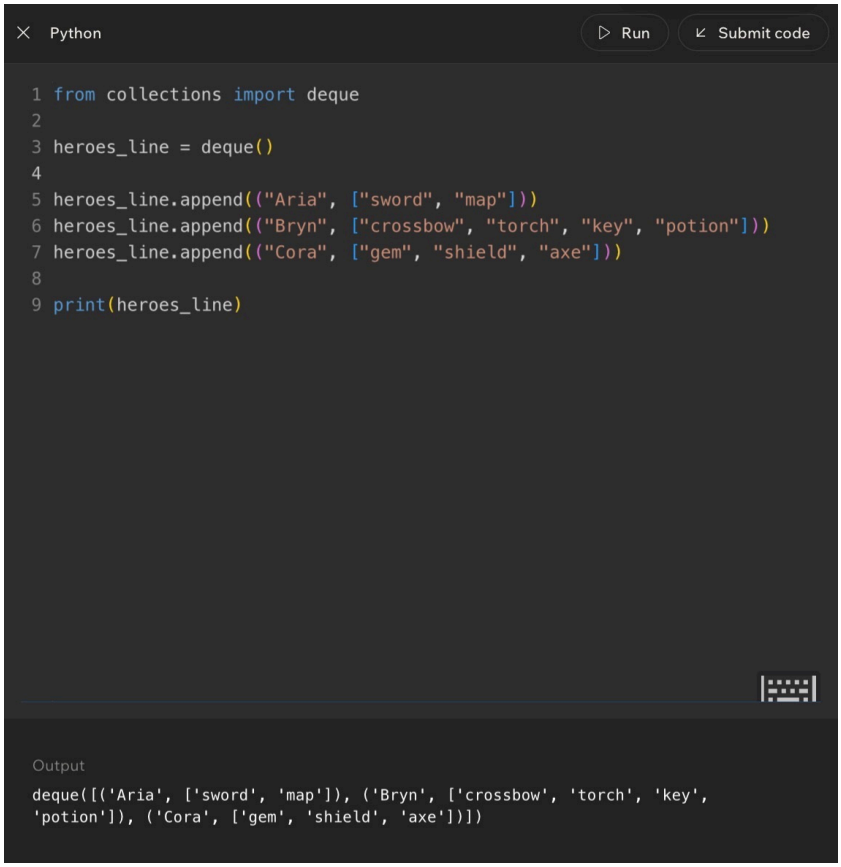
```
Current hero: Aria
Starting backpack: ['sword', 'map']
Aria uses map
Aria uses sword
```

Line-by-line explanation:

- heroes_line is a queue, so the first hero added is the first hero removed.
- Each hero is stored with a backpack list. That inner list acts like a stack.
- heroes_line.popleft() removes Aria because Aria is first in line.
- backpack.pop() removes Aria's last packed item first, so map is used before sword.

Real English translation: Let the first hero in line take a turn, then use that hero's backpack items from newest to oldest.

Code Example:



Made with Goodnotes

Lesson 12 - Integrating Stacks and Queues (Full Simulation)

Code Example 2 - Process Every Hero

```
from collections import deque

heroes_line = deque()
heroes_line.append(("Aria", ["sword", "map"]))
heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
heroes_line.append(("Cora", ["gem", "shield", "axe"]))

while heroes_line:
    current_hero, backpack = heroes_line.popleft()
    print("Current hero:", current_hero)

    while backpack:
        item = backpack.pop()
        print(current_hero, "uses", item)
```

What this code demonstrates:

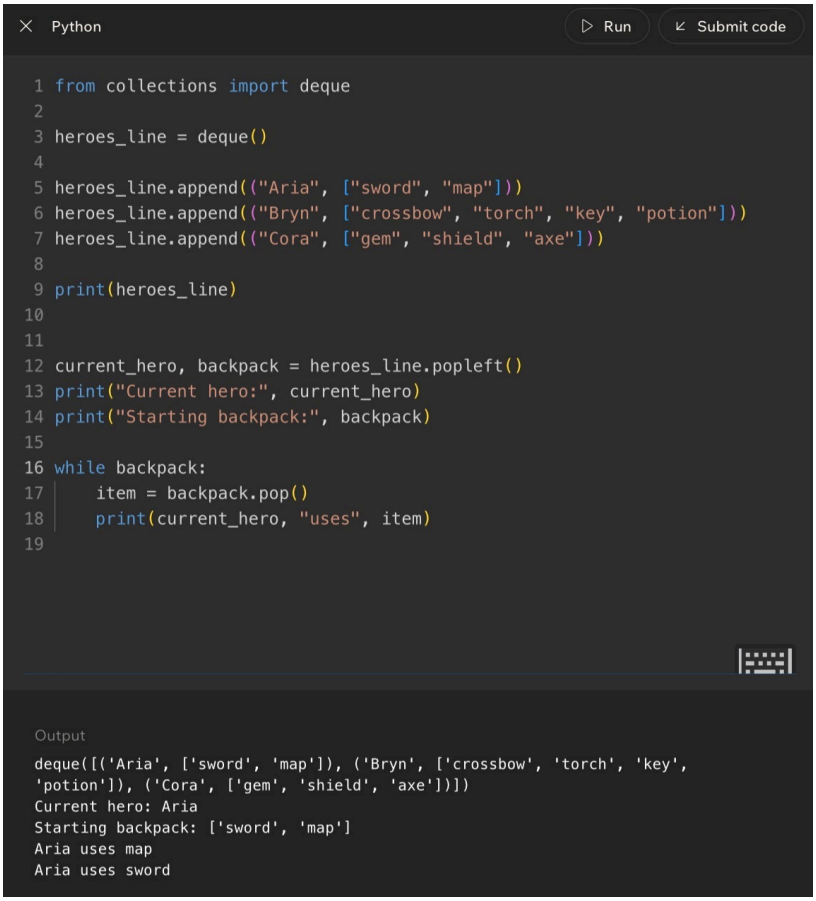
- The outer while loop is queue behavior. It processes heroes in the order they entered the line.
- The inner while loop is stack behavior. It processes each hero's items from last packed to first packed.
- The queue decides who goes next. The stack decides which item that hero uses next.

This is the real software-engineering lesson: one program can combine multiple data structures when different parts of the problem need different order rules.

Plain-English Algorithm

- While the hero line is not empty, remove the front hero from the queue.
- Give that hero a turn.
- While that hero's backpack is not empty, pop and use the top item.
- When the backpack is empty, move to the next hero in line.
- Stop when no heroes remain in the queue.

Code Example:



Made with Goodnotes

Lesson 12 - Integrating Stacks and Queues (Wrap-Up)

Common Mistakes and Helpful Tips

- Mistake: using one structure for everything. The hero line needs a queue, but the backpack needs a stack.
- Mistake: mixing up which operation belongs to which structure. popleft() is queue-front removal; pop() is stack-top removal.
- Mistake: forgetting that nested data has layers. First you remove a hero from the queue, then you work through that hero's backpack.
- Mistake: using pop() on the queue of heroes. That would take the newest hero instead of the oldest waiting hero.
- Helpful tip: write the behavior in English before choosing operations. If the phrase is 'first in line,' think queue. If the phrase is 'last packed,' think stack.
- Memory hook: queue chooses the turn order; stack chooses the most recent item.

Key Takeaways

- Stacks follow LIFO: Last In, First Out.
- Queues follow FIFO: First In, First Out.
- append() can add to the rear of a queue or the top of a stack depending on how the structure is being used.
- pop() removes the last item from a list, so it fits stack behavior.
- popleft() removes the first/front item from a deque, so it fits queue behavior.
- A single program can use both structures when different parts of the problem require different order rules.

Notes

Code Example:

Python

Run | Submit code

```
from collections import deque

heroes_line = deque()

heroes_line.append(("Aria", ["sword", "map"]))
heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
heroes_line.append(("Cora", ["gem", "shield", "axe"]))

print(heroes_line)


current_hero, backpack = heroes_line.popleft()
print("Current hero:", current_hero)
print("Starting backpack:", backpack)

heroes_line = deque()

heroes_line.append(("Aria", ["sword", "map"]))
heroes_line.append(("Bryn", ["crossbow", "torch", "key", "potion"]))
heroes_line.append(("Cora", ["gem", "shield", "axe"]))

while backpack:
    item = backpack.pop()
    print(current_hero, "uses", item)

while heroes_line:
    current_hero, backpack = heroes_line.popleft()
    print("== Hero:", current_hero, "==")
    while backpack:
        item = backpack.pop()
        print(current_hero, "uses", item)

```

Output

```
deque([('Aria', ['sword', 'map']), ('Bryn', ['crossbow', 'torch', 'key', 'potion']), ('Cora', ['gem', 'shield', 'axe'])])
Current hero: Aria
Starting backpack: ['sword', 'map']
Aria uses map
Aria uses sword
== Hero: Aria ==
Aria uses map
Aria uses sword
== Hero: Bryn ==
Bryn uses potion
Bryn uses key
Bryn uses torch
Bryn uses crossbow
== Hero: Cora ==
Cora uses axe
Cora uses shield
Cora uses gem
```

Made with Goodnotes

Week 2 Review Tools - Stacks and Queues

Master Framework

Week 2 is about order rules. A data structure is not just a container; it is a rule for what item can be added, accessed, or removed next. Stacks and queues both store collections, but they answer different order questions.

A stack asks: what was added most recently? A queue asks: what has been waiting the longest? That difference controls the entire week. If you can identify which end matters, the code usually becomes much easier to choose and trace.

- Stack = LIFO = Last In, First Out.
- Queue = FIFO = First In, First Out.
- Stack operation pair: push with append(), pop with pop().
- Queue operation pair: enqueue with append(), dequeue with pop(0) for a list or popleft() for a deque.

Why This Week Matters

Stacks and queues are small ideas with big consequences. Undo buttons, browser back history, function call stacks, print queues, task schedulers, customer service tickets, and event processing all depend on choosing the right removal order.

The programmer habit underneath the vocabulary is: before writing code, decide what item should leave next. If the newest item leaves next, use stack thinking. If the oldest waiting item leaves next, use queue thinking. If different parts of the program need different rules, combine the structures carefully.

One-Sentence Memory Hook

Stacks are piles. Queues are lines. pop() takes the newest from a stack. popleft() serves the oldest from a queue.

Made with Goodnotes

Stack Review - LIFO Thinking

Full Teaching Review

A stack is a collection where the most recently added item is the next one removed. The physical image is a pile of plates, books, folded towels, or boxes. You can easily reach the top item, but you cannot grab something from the middle without disturbing the stack.

In Python list practice, the top of the stack is the end of the list. Calling append() pushes a new item onto the top. Calling pop() removes and returns the current top item. After pop() runs, the stack has changed; the removed item is no longer inside the list.

The biggest stack skill is tracing state. You should be able to read a sequence of append() and pop() calls and predict both what value gets removed and what remains afterward.

Stack Operation Pattern

```
stack = []

stack.append("A")
stack.append("B")
stack.append("C")

removed = stack.pop()

print(removed)
print(stack)
```

Expected output:

```
C
['A', 'B']
```

Real English translation: Put A down first, then B, then C. Since C was added last, C is on top. pop() removes C and leaves A and B behind.

Stack Mistakes to Watch

- Thinking pop() removes the first item. It does not. On a list, pop() removes the last item by default.
- Forgetting that a pop operation changes the stack permanently.
- Trying to pop from an empty list without checking first.
- Calling a situation LIFO when the oldest item leaves first. That is queue/FIFO behavior, not stack behavior.

Made with Goodnotes

Queue Review - FIFO Thinking

Full Teaching Review

A queue is an ordered collection where the first item added is the first item removed. The physical image is a coffee shop line, food-truck line, checkout line, support ticket line, or pickup counter. New items join at the rear, and the next item served is taken from the front.

The front and rear are the only two directions you really need. The front is where items leave. The rear is where items enter. If the problem is about fairness, arrival order, or processing work in the order it arrived, it probably wants a queue.

In early examples, a regular list can imitate a queue with append() and pop(0). Later, deque becomes the better structure because popleft() expresses front removal clearly and handles queue-style removal more efficiently.

Queue Operation Pattern

```
from collections import deque

queue = deque()
queue.append("Sophia")
queue.append("CJ")
queue.append("Caleb")

served = queue.popleft()

print(served)
print(queue)
```

Expected output:

```
Sophia
deque(['CJ', 'Caleb'])
```

Real English translation: Sophia arrived first, so Sophia leaves first. CJ and Caleb are still waiting in the same order.

Queue Mistakes to Watch

- Using pop() on a queue. That removes from the rear and accidentally creates stack-like behavior.
- Forgetting to import deque before using deque().
- Calling popleft() when the queue is empty.
- Mixing up front and rear. New items enter at the rear; old items leave from the front.

Made with Goodnotes

Stack vs Queue Decision Guide

Core Comparison

Question	Stack Answer	Queue Answer
What item leaves next?	Newest item	Oldest waiting item
Order rule	Last In, First Out	First In, First Out
Physical model	Pile of plates	Line at a counter
Python removal	pop()	popleft() with deque, or pop(0) with list
Good for	Undo, browser back, call stack, nested items	Tickets, lines, print jobs, task processing
Common trap	Forgetting pop removes the last item	Using pop instead of popleft

How to Choose During a Quiz

- If the wording says most recent, latest, top, undo, last added, or last packed, think stack.
- If the wording says first arrived, fair order, line, waiting, ticket, request, or oldest task, think queue.
- If the code uses stack.pop(), expect the end of the list to disappear.
- If the code uses queue.popleft(), expect the front/left item to disappear.
- If the code combines a queue and a stack, identify the outer structure first, then trace the inner structure.

Out Loud Programmer Thinking

I do not start by memorizing the method name. I start by asking what behavior the real-world situation needs. If people should be served in arrival order, the structure is a queue. If the most recent action should be reversed first, the structure is a stack. After the behavior is clear, the Python method becomes easier to remember.

Made with Goodnotes

Trace Practice - Predict the State

Practice 1 - Stack Trace

```
stack = []
stack.append("red")
stack.append("blue")
stack.append("green")

first = stack.pop()
second = stack.pop()

print(first)
print(second)
print(stack)
```

Answer:

```
green
blue
['red']
```

Why: red entered first, blue entered second, and green entered last. A stack removes the newest item first, so green leaves, then blue, and red remains.

Practice 2 - Queue Trace

```
from collections import deque

q = deque()
q.append("T1")
q.append("T2")
q.append("T3")
q.append("T4")

first = q.popleft()
second = q.popleft()

print(first)
print(second)
print(q)
```

Answer:

```
T1
T2
deque(['T3', 'T4'])
```

Why: queues remove from the front. T1 entered first and leaves first. T2 becomes the new front and leaves second.

Made with Goodnotes

Weekly Review Question Breakdown

How the Review Questions Are Testing You

The weekly review is not really testing whether you can recite definitions in isolation. It is testing whether you can recognize order rules in real situations, map those rules to stack or queue vocabulary, and trace Python operations correctly.

For each question, underline the clue word. Words like top, recent, undo, and last added point toward stack/LIFO. Words like line, first arrived, waiting, support ticket, and fair order point toward queue/FIFO.

Question Type Guide

Question Type	What It Checks	Strategy
Real-world LIFO	Can you spot stack behavior?	Look for newest/top item leaving first.
Push operation	Do you know how stacks add items?	push means add to the top.
Pop trace	Can you predict the top after removal?	Label stack order before and after pop().
Undo behavior	Can you connect recent actions to stacks?	Undo removes the most recent action first.
Real-world FIFO	Can you spot queue behavior?	Look for first arrival served first.
Enqueue operation	Do you know where queues add?	enqueue adds to the rear/end.
Stack + queue blanks	Can you combine both rules?	Queue for turn order; stack for last-added inner items.
Support queue trace	Can you process tickets correctly?	Append in arrival order, then dequeue from the front.

Review Tip

When a question includes code, do not answer from vibes alone. Write the items in order, mark front/top, then cross out each removal. Yes, it feels extra. No, the quiz does not care about our feelings. Rude, but educational.

Made with Goodnotes

Common Exam Traps and Debug Thinking

Trap 1 - pop() vs pop(0) vs popleft()

These three removals look similar, but they do different things. pop() removes the last item from a list. pop(0) removes the first item from a list. popleft() removes the left/front item from a deque.

Expression	Removes	Behavior
items.pop()	Last item	Stack/LIFO
items.pop(0)	First item	Queue/FIFO with list
queue.popleft()	Left/front item	Queue/FIFO with deque

Trap 2 - Correct Structure, Wrong End

A lot of wrong answers come from using the right structure name but removing from the wrong end. A queue is only FIFO if the front item leaves first. A stack is only LIFO if the most recent item leaves first.

- If your queue removes from the rear, it is acting like a stack.
- If your stack removes from the bottom, it is not normal stack behavior.
- If you forget the empty check, your logic may work for normal examples but break on edge cases.

Debugging Checklist

- Step 1: Identify whether the problem is asking for LIFO or FIFO.
- Step 2: Label the top for stacks or front/rear for queues.
- Step 3: Trace each append, pop, pop(0), or popleft call in order.
- Step 4: After every removal, update the collection before reading the next line.
- Step 5: Check empty-collection cases before removing.

Made with Goodnotes

Week 2 Cheat Sheet

Syntax Reference

```
# Stack with a list
stack = []
stack.append("A")       # push onto top
stack.append("B")
top = stack.pop()        # removes "B"

# Queue with a deque
from collections import deque
queue = deque()
queue.append("A")       # enqueue at rear
queue.append("B")
front = queue.popleft()  # removes "A"
```

Vocabulary Reference

Term	Meaning
LIFO	Last In, First Out. The newest item leaves first.
FIFO	First In, First Out. The oldest waiting item leaves first.
Stack	A collection that behaves like a pile. Use append and pop.
Queue	A collection that behaves like a line. Use append and popleft with deque.
Push	Add an item to the top of a stack.
Pop	Remove and return the top item of a stack.
Enqueue	Add an item to the rear of a queue.
Dequeue	Remove and return the front item of a queue.
deque	Python's double-ended queue structure from collections.

Made with Goodnotes

Final Week 2 Memory Sheet

If You Remember Nothing Else

- Stacks remove the most recently added item first.
- Queues remove the earliest added item first.
- append() adds to the end of a list or deque.
- pop() removes the end of a list, which fits stack behavior.
- popleft() removes the front/left side of a deque, which fits queue behavior.
- Always trace state after every removal. The collection changes.
- Always consider what happens if the structure is empty.

Final Concept Map

Week 2 can be reduced to one chain of ideas:

Data structures organize behavior. Stacks organize newest-first behavior. Queues organize oldest-first behavior. Python lists can model stacks easily. Python deque handles queues more cleanly. Real programs often combine structures when one part needs FIFO and another part needs LIFO.

Final Reflection

The deeper lesson is that programming is not just about storing data. It is about preserving the meaning of the data. If the order matters, the structure must protect that order. That is why stacks and queues show up everywhere: they let code model real-world rules in a reliable way.

Notes

Made with Goodnotes